



Greedy Algorithms

Local Optimization → Global Optimization?

Java 8+



Session Structure

1. What is Greedy?
2. When does Greedy work?
3. Greedy vs Dynamic Programming
4. Classic Greedy Problems
5. Real-world applications
6. Complexity analysis
7. Interview strategies



What is a Greedy Algorithm?

Greedy strategy:

Make the best local choice at each step.

Hope it leads to global optimum.

Key idea:

Never reconsider previous decisions.



When Does Greedy Work?

Greedy works when:

1. Greedy choice property
2. Optimal substructure

If not satisfied → Greedy may fail.

Greedy vs Dynamic Programming

Feature	Greedy	DP
Revisits choices	No	Yes
Complexity	Usually lower	Higher
Guarantees optimal?	Not always	Yes (if designed properly)



Activity Selection Problem

Select max non-overlapping activities.

Strategy:

Sort by finish time.

Time: $O(n \log n)$

Activity Selection Code

```
static int activitySelection(int[] start, int[] end) {  
    int n = start.length;  
    int[][] activities = new int[n][2];  
  
    for (int i = 0; i < n; i++) {  
        activities[i][0] = start[i];  
        activities[i][1] = end[i];  
    }  
  
    java.util.Arrays.sort(activities,  
        java.util.Comparator.comparingInt(a -> a[1]));  
  
    int count = 1;  
    int lastEnd = activities[0][1];  
  
    for (int i = 1; i < n; i++) {  
        if (activities[i][0] >= lastEnd) {  
            count++;  
            lastEnd = activities[i][1];  
        }  
    }  
    return count;  
}
```



Fractional Knapsack

Take items with highest value/weight ratio first.

Time: $O(n \log n)$

THIRAVUGAL

Fractional Knapsack Code

```
static double fractionalKnapsack(int[] value,
                                int[] weight, int capacity) {

    int n = value.length;
    double[][] items = new double[n][2];

    for (int i = 0; i < n; i++) {
        items[i][0] = (double)value[i] / weight[i];
        items[i][1] = i;
    }

    java.util.Arrays.sort(items,
        (a, b) -> Double.compare(b[0], a[0]));

    double totalValue = 0;

    for (int i = 0; i < n && capacity > 0; i++) {
        int idx = (int)items[i][1];

        if (weight[idx] <= capacity) {
            totalValue += value[idx];
            capacity -= weight[idx];
        } else {
            totalValue += items[i][0] * capacity;
            capacity = 0;
        }
    }

    return totalValue;
}
```



Coin Change (Greedy Works Only Sometimes)

Example:

Coins: 1, 5, 10, 25

Greedy works.

But:

Coins: 1, 3, 4

Greedy fails.

Greedy not universally correct.



Minimum Number of Platforms

Sort arrivals & departures.

Time: $O(n \log n)$

THIRAVUGAL



Gas Station Problem

Determine if you can complete circuit.

Time: $O(n)$

THIRAVUGAL

Gas Station Code

```
static int canCompleteCircuit(int[] gas, int[] cost) {  
    int total = 0, tank = 0, start = 0;  
  
    for (int i = 0; i < gas.length; i++) {  
        int diff = gas[i] - cost[i];  
        total += diff;  
        tank += diff;  
  
        if (tank < 0) {  
            tank = 0;  
            start = i + 1;  
        }  
    }  
  
    return total >= 0 ? start : -1;  
}
```



Huffman Coding

Used in:

- File compression
- ZIP
- JPEG

Uses Greedy:

Combine smallest frequency nodes first.

Time: $O(n \log n)$



Job Scheduling with Deadlines

Maximize profit.

Strategy:

Sort by profit descending.



Real World Use Case — Network Routing

Minimum Spanning Tree (Prim's Algorithm).

Uses Greedy.

Time: $O(E \log V)$



Real World Use Case — Load Balancing

Assign highest priority tasks first.

Used in:

- CPU scheduling
- Distributed systems



Why Greedy is Fast?

Because:

- No backtracking
- No revisiting states
- Often $O(n \log n)$

THIRAVUGAL

Complexity Summary

Problem	Time
Activity Selection	$O(n \log n)$
Fractional Knapsack	$O(n \log n)$
Gas Station	$O(n)$
Huffman	$O(n \log n)$

Common Mistakes

- Applying greedy blindly
- Not verifying greedy choice property
- Confusing 0/1 knapsack with fractional
- Ignoring sorting step



Interview Strategy

Ask:

1. Can I make a locally optimal choice?
2. Does it guarantee global optimal?
3. Do subproblems overlap?
4. Is sorting required?

If unsure → consider DP.



Greedy Proof Strategy

To prove greedy correctness:

1. Exchange argument
2. Contradiction proof
3. Show optimal substructure



Performance Thought Exercise

Dataset size = 1 million.

DP:

$O(n^2)$ → too slow.

Greedy:

$O(n \log n)$ → scalable.



Summary

Greedy algorithms:

- Make local optimal decisions
- Often very efficient
- Work when greedy choice property holds
- Used in scheduling, compression, networking

Master:

- Activity Selection
- Fractional Knapsack
- Huffman Coding
- Gas Station
- Job Scheduling